

# Satellite Simulation Workshop

## Introduction

Dr Simon Lock

# Aims of this Session

Appreciate complexity of "Computer Based Systems"

Software + Hardware + People + Processes

With a focus on role of human factors in such systems

This will be achieved through:

- The use of purpose-build software simulation
- Investigation of a specific historical case study
- Exploring discussion from academic publications

# Computer Science

In many complex applications, Computer Science often works in collaboration with other disciplines

Computers are SO useful, everyone is using them !  
This however requires knowledge of BOTH disciplines  
("Computer Science" AND the "Application Domain")  
Not two separate people, but individuals with BOTH

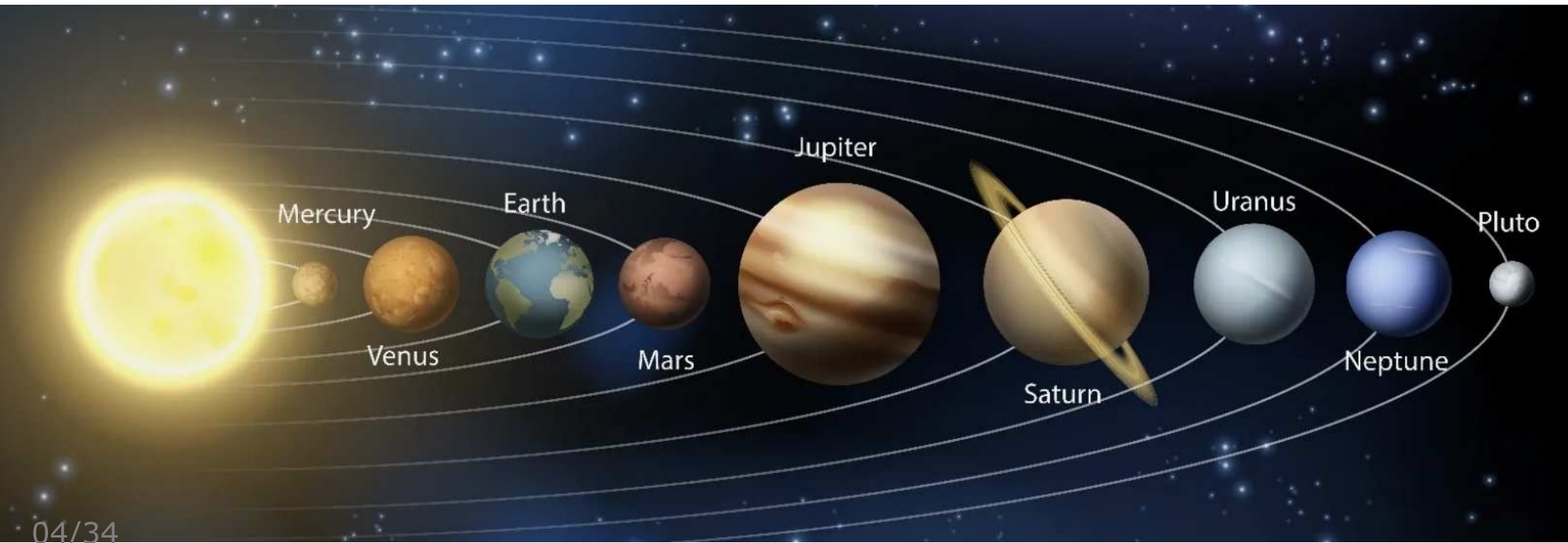
Let's look at one such application domain...

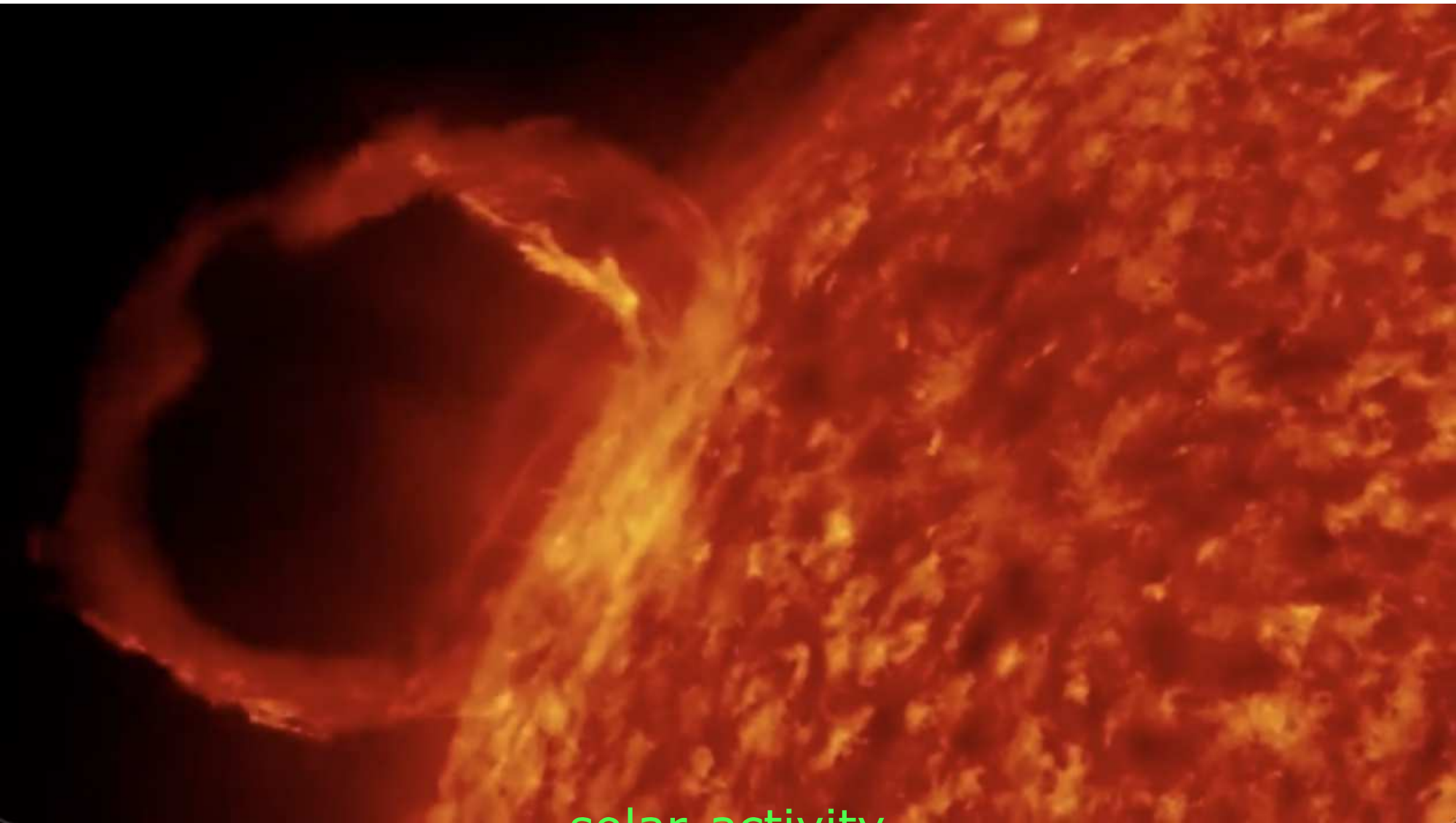
# Our Solar System

Let's test your knowledge of our solar system

How many planets are there (8/9/10) ?

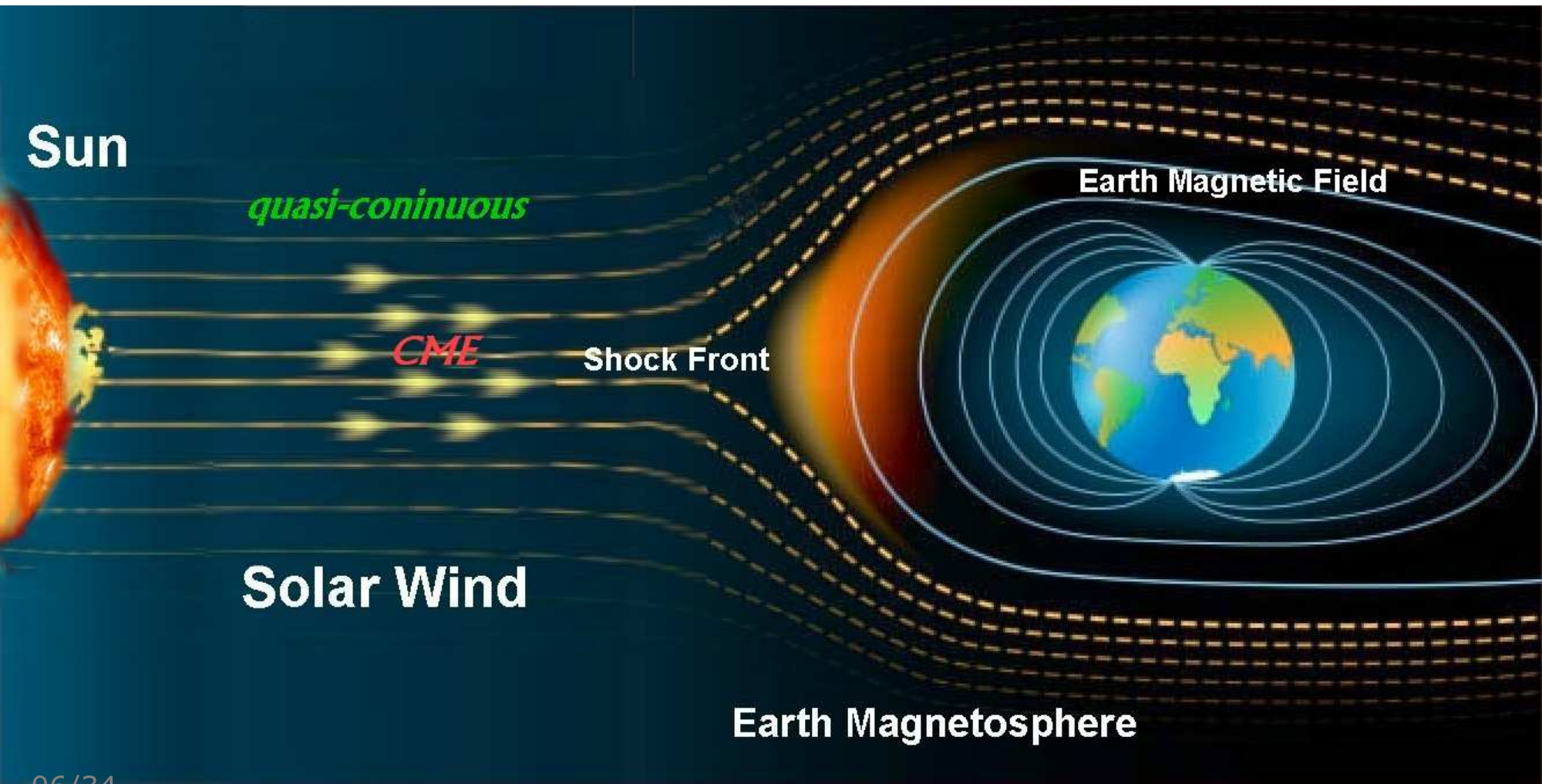
Which Orbits Which





solar-activity

# Solar Wind



# Aurora Borealis - Northern Lights



# Solar Wind

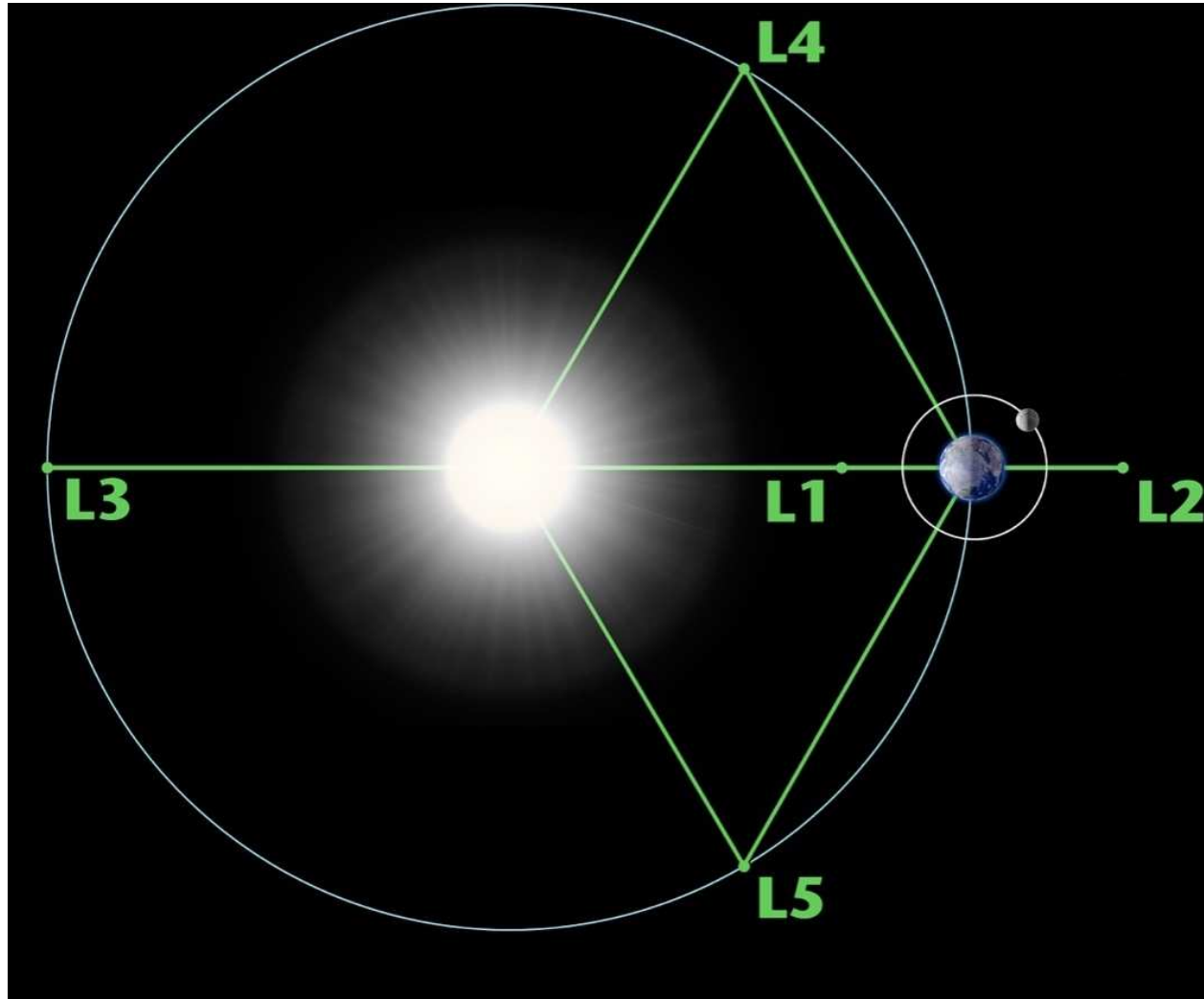
Travels at around 400 km/s ( $\sim 1.5$  million km/h)  
(yet still takes three or four days to reach Earth !)

When it reaches the Earth, solar wind can:

- cause power outages on the ground
- interfere with communication networks
- disrupt the operation of satellites
- interfere with navigation systems
- injure astronauts in space !

A good idea to use satellites to monitor !

# Lagrange Points (not to scale !)



# Lagrange Point "L1"

Lagrange Point "L1" is between the Sun and Earth  
An ideal location for monitoring risk of solar wind !

L1 is 1.5 million km from the Earth

For reference, Earth is approx.  
150 million km from Sun (99:1)

Takes about 4 months to travel  
from Earth to L1 (at 500 km/h)

# Satellites currently at L1

Solar and Heliospheric Observatory (SOHO)

Aditya-L1 (arrived in January 2024)

Global Geospace Science "Wind" satellite

Deep Space Climate Observatory (DSCOVR)

Advanced Composition Explorer (ACE)...



<http://services.swpc.noaa.gov/text/ace-swepam.txt>

# Graphing

It can be hard to interpret raw numerical data  
Instead we can pull the live feed from the satellite  
And represent the numerical data visually:

ACEGrapher

# Practical Activity

Enough theory and high-level discussion  
Let's get hands-on with a practical activity

## UNFORTUNATELY

We don't have a real satellite to play with :o(

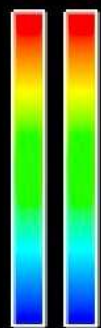
## FORTUNATELY

Since we are Computer Scientists...  
We can \*build\* a simulator to experiment with !



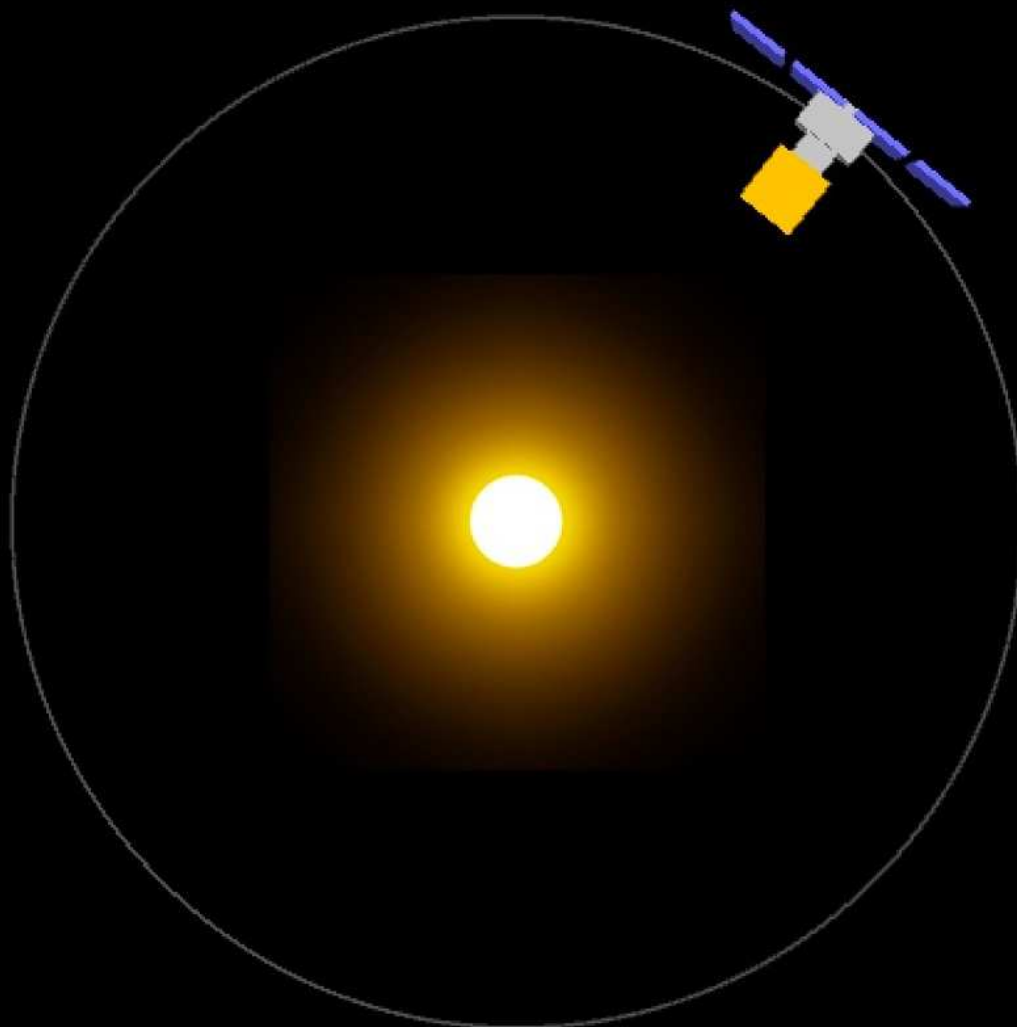
xyz:0.6800769,-0.7331408,-0.35727394 fti:0.511005 ftt:1.0 bat:1.0000614  
xyz:0.6800769,-0.7331408,-0.35727394 fti:0.511005 ftt:1.0 bat:1.0000614

Temp



Tank  
Pipe

- SUMER
- VIRGO
- GOLF
- COSTEP
- ERNE
- UVCS
- MDI
- LASCO
- EIT
- CDS
- SWAN
- CELIAS



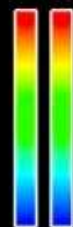
Fuel

SEND



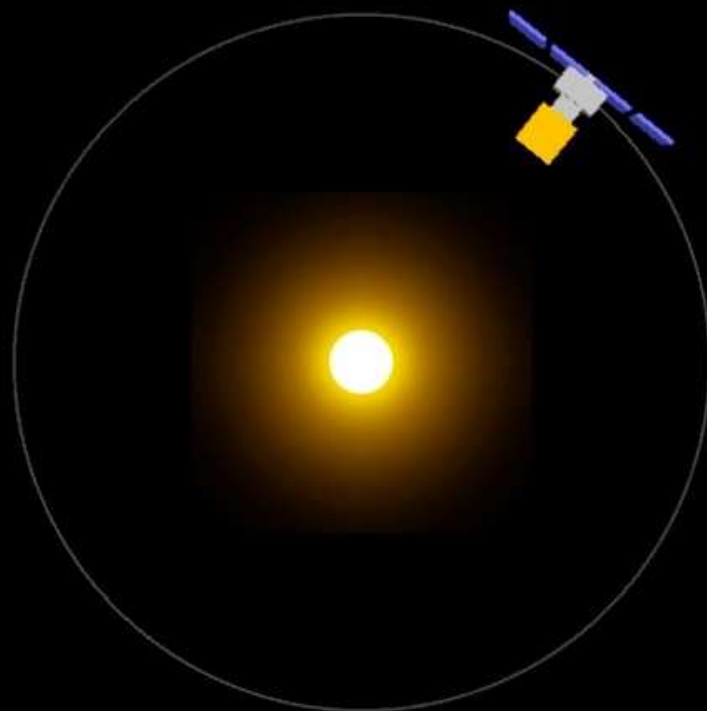
xyz: 0 6800769, -0 7331408, -0 35727394 fti: 0 511005 ftt: 1 0 bat: 1 0000614  
xyz: 0 6800769, -0 7331408, -0 35727394 fti: 0 511005 ftt: 1 0 bat: 1 0000614  
xyz: 0 6800769, -0 7331408, -0 35727394 fti: 0 511005 ftt: 1 0 bat: 1 0000614

Temp



Tank  
Pipe

- SUMER
- VIRGO
- GOLF
- COSTEP
- ERNE
- UVCS
- MDI
- LASCO
- EIT
- CDS
- SWAN
- CELIAS



SEND



Fuel

# Important Note

Note that this simulation is an \*approximation\*  
(not everything is scientifically accurate)

We've made a few compromises to make it usable:

- Scale: can't fit 150 million km on the screen !
- Time: sped up animation to reduce waiting time
- Dimensions: 2D rather than 3D for easier interaction
- Simplified Commands: to make more understandable

# Simulator downloaded

Simulator written in Processing

But exported as an application !

<https://tinyurl.com/BristolSun>

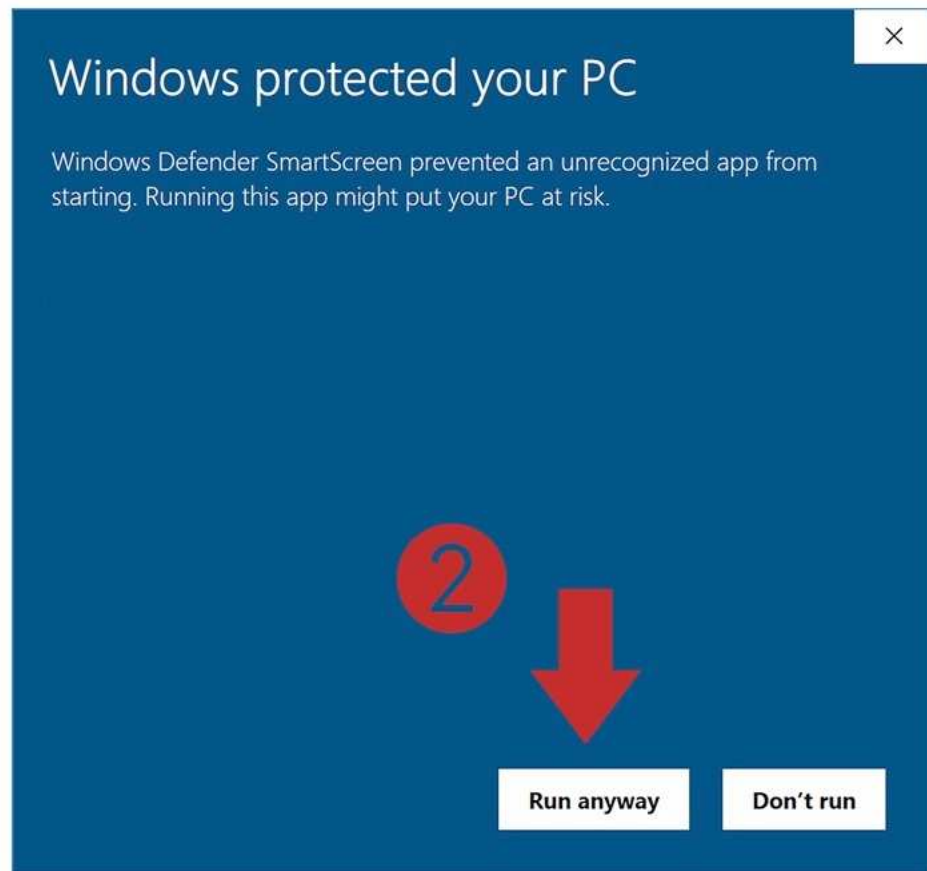
Download it, UNZIP IT, run it:

`SatelliteController`

Will need to manage security

Depending on your platform...

# Windows Security



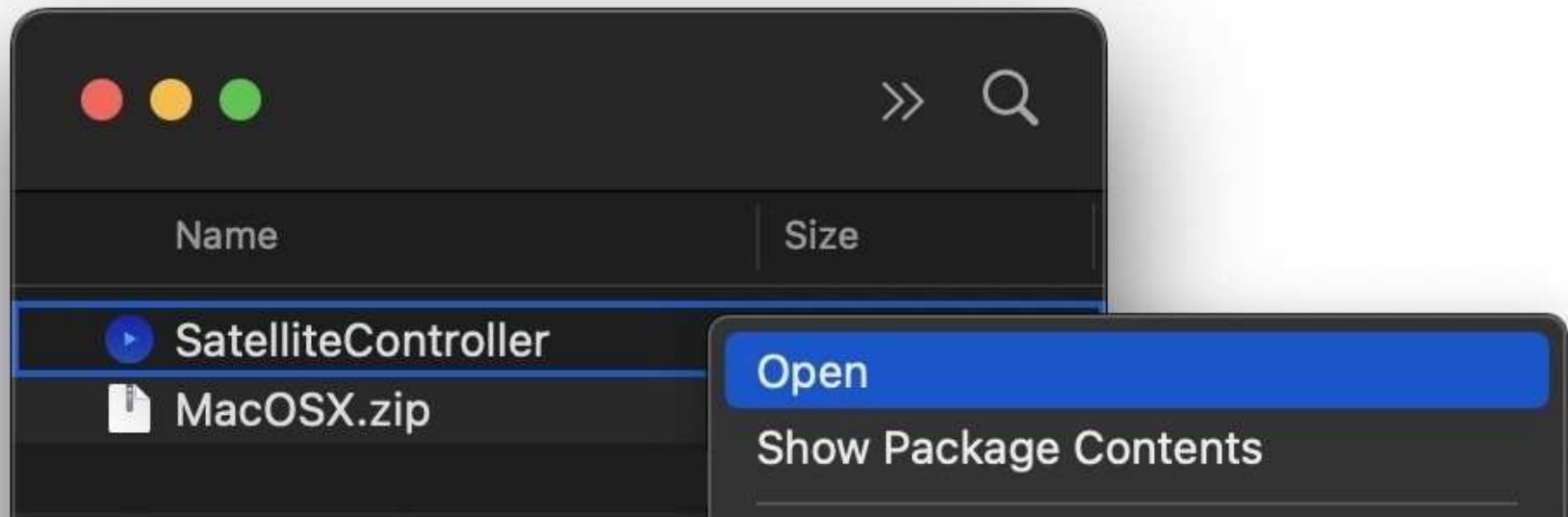
# MacOSX Security

Download then Unzip MacOSX in Downloads folder

Important: move SatelliteController to suitable folder

Control-Click on the SatelliteController file

Then select "Open" from the menu



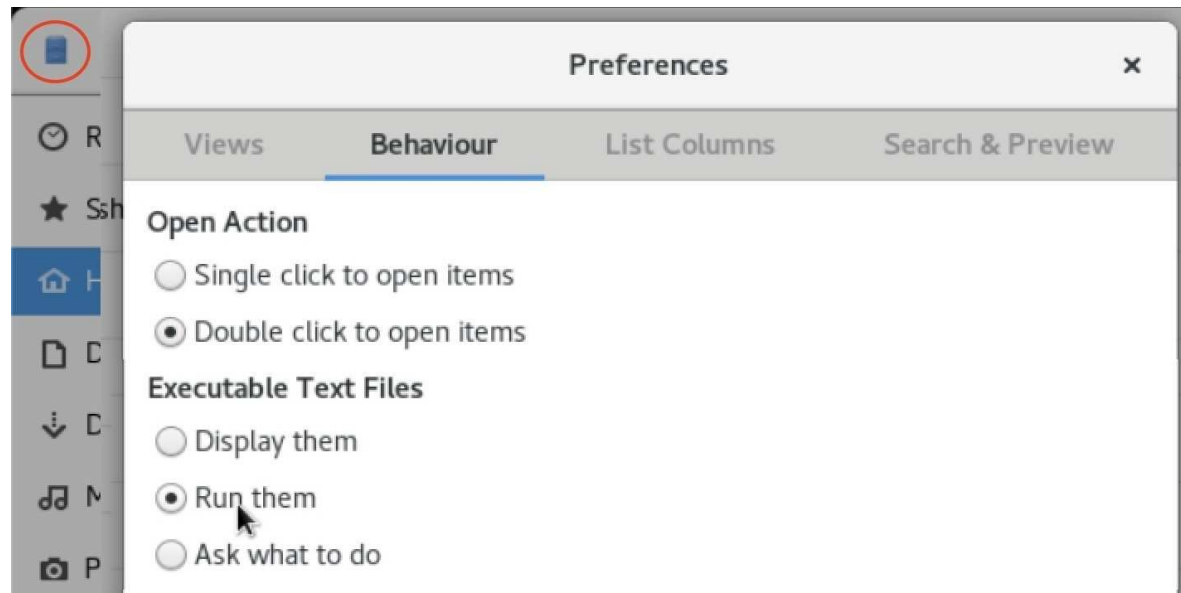
# Linux Security (including lab machines)

Open up the terminal and then type "nautilus"

When file explorer opens, go to "preferences" menu

For "Behaviour > Exec Text Files" select "Run them"

You can now run SatelliteSimulator by double-clicking



These installation slides are on GitHub

<https://github.com/UoB-SummerSchool/>

Look for the "SatelliteSimulation" repository

[ Wait for everyone to start their simulator ]

A range of control commands are available  
Let's open up the simulation to try them out:

## SatelliteController

Run your simulator and follow along on your computer  
As we step through commands on following slides...

# Science Experiments

Satellite carries a range of scientific experiments  
We won't explore the nature of these experiments  
(they are space science, this is computer science)

We switch OFF/ON all science experiments using:

SCI0

SCI1

Useful if we need to save some power, however...  
Scientists won't be very happy if we leave them off

# WARNING

We are about to use some navigation commands

It is ESSENTIAL that we are VERY careful with these

We MUST NOT let satellite get out of alignment

Move too much and we will lose solar power collection

May also lose radio signal (due to misaligned antenna)

## BE VERY CAREFUL

Only move satellite by small amounts

Always move it back again afterwards

# Navigation

Before navigation, we must switch on Gyroscope

GYR1

We can then change the "pitch" of the satellite

PIT15

PIT-20

As well as the "yaw"

YAW-20

YAW10

Try these out now, but...  
careful with fuel consumption!

# Sun Reacquisition

Due to all our previous manual navigation actions...

The satellite can quickly become out-of-alignment:

- Alignment with the Sun (for solar power)
- Alignment with the Earth (for radio control signals)

Luckily satellite provides an "auto pilot" feature:  
"Emergency Sun Reacquisition"

ESR

Resets the satellite to the correct orientation

Try running this command now